
VOLT: Vision and Language Trajectory Segmentation for Faster-than-Demonstration Policies

Robert Ramirez Sanchez¹ · Daniel J. Evans¹ · Dylan P. Losey¹ · Siddarth Jain²

Abstract Humans often take longer to demonstrate a task than a robot would need to execute it. Rather than learning to replicate the demonstration at the same pace, many industrial and practical applications require robots to perform tasks as quickly as possible. In this paper, we investigate several hypotheses for learning policies that operate *faster-than-demonstrations*. Our experiments show that the most effective strategy is to downsample recorded demonstrations and train the robot’s policy on this accelerated data. However, uniformly downsampling an entire trajectory can be problematic. Some parts of a task can be safely sped up (e.g., unconstrained motion), while others demand slower, more precise motion (e.g., object interactions or fine manipulation). To address this challenge, we introduce VOLT, a vision-and-language trajectory segmentation method that reasons over video demonstrations, and leverages contextual cues to determine when acceleration is appropriate and when careful precision is required. VOLT identifies segments where slow, deliberate motion is necessary, then selectively downsamples the remaining segments. The resulting reformat-

ted trajectories can be used with standard imitation learning approaches, such as diffusion policies. Our results highlight that segmentation quality is critical—baseline methods often misidentify when acceleration is possible, leading to overly cautious or unreliable policies. Compared to state-of-the-art alternatives, VOLT allows robots to execute tasks faster while maintaining strong performance. See our project website: <https://volt2026.github.io/VOLT2026/>

Keywords Learning from Demonstrations, Imitation Learning, Semantic Scene Understanding, Deep Learning in Grasping and Manipulation

1 Introduction

Robots should be capable of performing tasks with precision and speed. Recent works have enabled robots to learn complex tasks by imitating human experts [33, 16, 28]. Since the quality of demonstrations dictates the performance of imitation learning approaches [31, 3, 8], operators tend to demonstrate tasks at a lower speed to minimize errors. Indeed, even if the human demonstrates at a normal speed — robots often learn to perform tasks slower than demonstrated. But autonomous systems are capable of completing tasks significantly faster than human demonstrators, and this acceleration is beneficial for industrial applications. So how can robots take a given demonstration and speed-up their learned policy?

Existing works often attempt to accelerate learned policies during either training or execution. For example, *training* approaches include high-frequency controllers that reduce teleoperation time [35], as well as data augmentation methods that resample the provided demonstrations [22, 14]. At the other extreme, *execution* approaches accelerate a learned policy by leveraging

Robert Ramirez Sanchez
E-mail: robertjrs@vt.edu

Daniel J. Evans
E-mail: daniellevans@vt.edu

Dylan P. Losey
E-mail: losey@vt.edu

Siddarth Jain
E-mail: sjain@merl.com

¹ Mechanical Engineering Department, Virginia Tech, Blacksburg, USA

² Mitsubishi Electric Research Laboratories (MERL), Cambridge, MA, USA

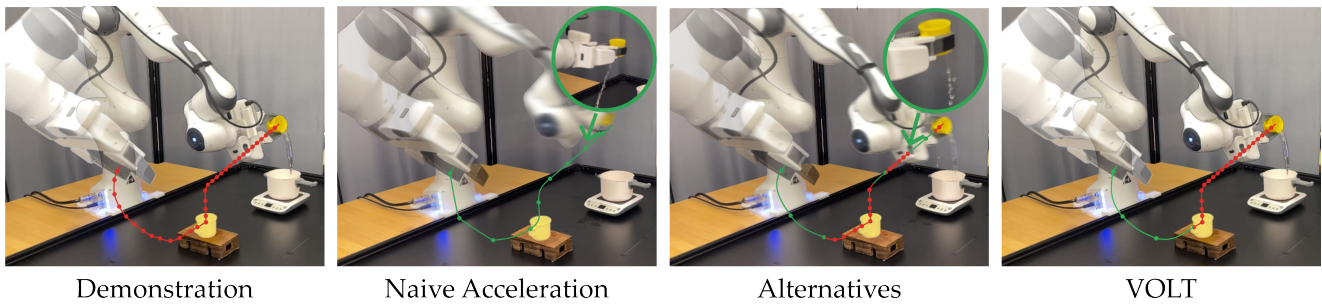


Fig. 1: (Left) A human shows the robot how to carry a cup full of water and then pour that water into a pot. (Middle) Naively accelerating the entire demonstration causes the robot to move too quickly (faster motions shown in **green**), accidentally spilling water as it moves. Alternatives slow down the robot in segments (slower motions shown in **red**). But these segments are often imperfectly chosen, and the robot still accelerates and spills water while carrying the cup. (Right) VOLT addresses this problem by using context cues from the entire video demonstration to accelerate only the parts of the demonstration that do not require precision.

high-fidelity controllers to precisely track the original behavior at variable frequencies [1]. Both types of approaches recognize two key problems. First, the robot needs to decide *how* to accelerate its policy, and second, the robot must determine *which parts* of its trajectory are appropriate to accelerate. Speeding up the motion often leads to failures, particularly when the robot must perform precise subtasks. (Figure 1).

In this work we seek a fundamental understanding of both questions: (a) how do we determine which segments of a demonstration to accelerate, and (b) what approaches are most effective for accelerating those segments. We adopt an experimental approach, and test a variety of hypothesis across a standardized testing setup. Our insight — consistent with prior works — is that downsampling the human’s demonstration provides an effective method for speeding-up the learned policy. However, unlike prior works, we argue that high-level task knowledge is often necessary to determine which parts of the trajectory to accelerate. Specifically, we hypothesize that:

Reasoning over videos of the entire demonstration provides the context cues needed to decide when to accelerate.

We accordingly apply vision and language models (VLMs) to develop **VOLT**. The key idea for VOLT is that the robot collects its standard visual demonstration, and then passes that demonstration through a VLM to determine the correct segmentation. After determining which parts the robot can safely accelerate, VOLT then downsamples those segments and passes the refined dataset to the given imitation learning algorithm. This enables the robot to learn a policy that is faster than the human’s demonstration (e.g., speeding up to quickly reach the cup), but also effective at com-

pleting the task (e.g., slowing down to carefully grasp that cup). In summary, this work makes the following contributions:

Accelerating Robot Policies. We introduce VOLT, a framework for enabling faster-than-demonstration policies. To the best of our knowledge, VOLT is the first approach to use a VLM to holistically analyze human demonstrations without explicitly defined features, segmenting them into regions that can be safely accelerated and regions that require precise execution.

Determining How to Accelerate. We develop and test a variety of hypotheses for effective faster-than-demonstrator policies. These include accelerating policy execution at test-time and learning faster policies via train-time downsampling. Our results suggest that train-time downsampling is a more consistent approach, particularly when using diffusion policies.

Comparing to Baselines. We compare VOLT to state-of-the-art baselines for faster-than-demonstration policies across a diverse range of tasks. Overall, we find that VOLT achieves up to a x2.57 speedup over policies trained on raw demonstrations while maintaining similar success rates. We conclude with practical guidelines for designers who want to accelerate policies within an imitation learning context.

2 Related Work

Emerging work in imitation learning enables robots to complete complex and long-horizon tasks [6, 11, 29, 34]. In general, these methods focus on matching the action distribution of the human’s dataset. Recent research suggests that robots can improve their performance and

generalization abilities by learning task-relevant representations [24, 36], leveraging language-based reasoning for task planning [10, 15], and learning from large-scale robot datasets [19, 5]. But these methods primarily focus on task success — in this paper, we are interested in the task speed. Human demonstrations are typically slow to avoid mistakes, and even if the human moves quickly, for industrial applications we often want robots to operate beyond the human’s capabilities. Our VOLT framework is not tied to any specific imitation learning algorithm. But to provide results that are consistent with the state-of-the-art methods, we will leverage diffusion policies in our experiments [9].

Unintentional Speed Up. Within the context of imitation learning, several methods alter the given dataset to improve data efficiency and mitigate compounding errors [23, 25, 13, 4]. The common theme across these works is to reduce the prediction horizon required to complete a task. Although accelerated execution is not the objective, the changes these methods make to the demonstrations often result in a sped-up policy. For example, Belkhale *et al.* [4] propose a hybrid architecture that dynamically switches between waypoints and dense low-level actions; the robot moves more rapidly between the waypoints marked by the human teacher. Similarly, Shi *et al.* [25] introduce Automatic Waypoint Extraction (AWE), a method to automatically extract waypoints from trajectories via linear interpolation within a specified error threshold. A different perspective is to decompose the trajectory into a set of primitives, and then learn a policy to sequence these primitives — again, reducing the prediction horizon [13]. The insight we obtain from these works is that data alterations are an effective way to reach faster-than-demonstration policies. In VOLT we extend these ideas, while specifically focusing on how rapidly the learned policy performs the entire task.

Intentional Speed Up. Recent works seek to maximize the robot’s speed without significantly impacting its performance [22, 32, 14, 1, 18]. In reinforcement learning, existing works include learning speed interpolation policies [32], and performing online fine-tuning to select faster actions [22]. Although these approaches can capture complex speed-up strategies, they typically require millions of online interactions, making them impractical in most real-world tasks. Most similar to our approach are offline imitation learning methods that label each demonstration by classifying states suitable for faster task execution. Guo *et al.* [14] learn a proxy policy from the original dataset to estimate the conditional action entropy distribution. They denote high-entropy actions as “casual” segments and low-entropy

actions as “precise”. By contrast, [1] estimates motion complexity via AWE [25] waypoints and DBSCAN [12] clustering to identify precise segments. We hypothesize that these approaches can be inaccurate because they do not consider the high-level context of tasks. Towards this objective, [18] explicitly defines 3D gripper-object distances as high-level context extracted from images. They estimate these features using VLMs, then parse them into text for an LLM to label a single demonstration, and propagate these labels via Dynamic Time Warping. But there are human intents not encoded by these designer-chosen features, for example, gripper-object interaction (e.g., grasping vs. pushing), or manipulation of deformable bodies. In VOLT we do not assign intermediate features; instead, we directly use VLMs to reason over video demonstrations.

3 Problem Statement

We consider visual imitation learning from offline datasets of human demonstrations, where a robot arm is teleoperated by a human while the system records environment states and actions. Because the quality of the learned policy is correlated to demonstration fidelity, experts typically perform demonstrations slowly and cautiously to avoid mistakes. This results in policies that can achieve high task success but execute behaviors at relatively low speeds. The central problem we address is how to improve execution efficiency, specifically, increasing task execution speed without degrading task performance or safety. To this end, we aim to (a) autonomously identify which portions of a task can be safely executed at higher speeds, and then (b) develop a strategy for accelerating those segments without degrading overall performance.

Policy. Let $x \in \mathcal{X}$ represent the robot joint and gripper position, and let $y \in \mathcal{Y}$ denote the environment state observed through RGB cameras. We combine both the robot state and RGB images into observation $o_t = (x_t, y_t)$. The robot’s learned policy π maps these observations into actions:

$$A_t \sim \pi(\circ \mid O_t) \quad (1)$$

where $A_t = \{a_t, a_{t+1}, \dots, a_{t+H}\}$ is a sequence of actions with horizon H , and each action a_t is a commanded robot joint and gripper position. The observation history $O_t = \{o_{t-M}, \dots, o_t\}$ is composed of the most recent M observations.

In our experiments we will instantiate Equation (1) as a diffusion policy [9]. To match state-of-the-art performance, we will apply the Denoising Diffusion Im-

PLICIT Models approach [26]; this requires fewer denoising iterations during inference (as compared to training) for faster action generation. The inference process is defined by the following equation:

$$A_t^{k-1} = \alpha(A_t^k - \gamma \varepsilon_\theta(O_t, A_t^k, k) + \sigma), \sigma \sim \mathcal{N}(0, \sigma^2 I) \quad (2)$$

where ε_θ is the noise prediction network with learnable parameters θ that are optimized to predict the noise added to each sample. The noise scheduler hyperparameters α , γ , and σ control the signal scaling and the magnitude of Gaussian noise added to each iteration k .

Labeling Dataset. To train π the human provides a dataset of task demonstrations $\mathcal{D} = \{\xi_0, \xi_1, \dots, \xi_N\}$. Each demonstrated trajectory ξ is a sequence T of observation-action pairs: $\xi = \{((x_0, y_0), a_0), \dots, ((x_T, y_T), a_T)\}$. As we will show, learning a policy directly from $\mathcal{D}$ limits the robot’s speed to that of the human teacher. One potential way to accelerate the learned policy is to first relabel each demonstration $\xi = \{(\tau_0, l_0), \dots, (\tau_k, l_k)\}$ into ordered non-overlapping segments $\bigcap_{i=1}^k \tau_i = \emptyset$. The labels $l_i \in L = \{\text{speed-up}, \text{maintain-speed}\}$ denote whether the given segment can be executed faster than demonstrated, or whether the segment requires precision or dexterity (e.g., grasping, insertion) and therefore the policy should match the human’s original speed.

Hierarchical Control. Actions a indicate the desired joint position and gripper state. To execute the policy quickly, the robot must select control inputs that rapidly perform these actions. In our experiments we update the target robot position at 20 Hz (e.g., we output 20 actions a_t per second). We then reach these target positions using a low-level velocity controller that operates at 1 kHz. The specific controller depends on the robot (in our experiments, we tested with a Franka Emika robot arm). But we generally focus on problem settings where the robot has a high-level diffusion policy — outputting desired positions a — and a low-level controller — which can accurately track those targets.

4 How Do We Enable Faster-than-Demonstration Policies?

Our goal is to develop robot policies that maximize task speed while maintaining optimal performance. However, simply training a policy π on collected trajectories $\mathcal{D}$ will inherently limit the robot’s motion to the pacing of the human demonstrator. In this section, we therefore test different methods that designers might

intuitively consider for speeding up the robot’s actions. We divide these methods into two groups: accelerating the policy during *execution* (i.e., reducing the number of high-level actions to shorten the task), or accelerating the policy during *training* (i.e., downsampling the dataset to emulate a faster demonstration). Our experiments in Sections 4.1 and 4.2 suggest that the training paradigm is more effective for enabling faster policies and retaining the policy’s success rate.

After we collect this experimental data for faster-than-demonstration policies, we next need to determine which parts to accelerate. We find that naively speeding up *all* the robot’s actions exacerbates failures, particularly when the robot requires precise motion. We address this problem through our proposed VOLT algorithm in Section 4.3. VOLT reasons over video demonstrations and segments these trajectories into sections that should either be accelerated or left at the demonstrated speed. We downsample the marked segments and train the robot; our resulting policies yield faster execution while maintaining task performance. However, we note that there are practical limits on how much the robot can accelerate, even after correctly identifying these segments. In Section 4.4 we discuss the practical factors for maximizing speed.

Baseline Policy. We train diffusion policies using the original dataset (*Normal Speed*) as our baseline for performance and task speed. Diffusion policies are capable of accurately learning action chunks, which are crucial for speeding up execution. In our experiments, inference delay is about ~ 0.1 seconds to generate an action sequence of 32 actions. Since the inference time is greater than the desired 20 Hz high-level control frequency, we eliminate execution delays caused by sensing-inference delays through asynchronous inference, and enforce temporal consistency via Error-Adaptive-Guidance (EAG) [1]. During training, we utilize reached robot poses x_{t+1} instead of the commanded poses a_t as our policy actions because the low-level controller can track the former with minimal error. These additions to the standard diffusion policies were made to facilitate acceleration.

Acceleration. Within this section we will test two paradigms for faster-than-demonstration policies. These paradigms perform acceleration during either execution (predicted action downsampling) or during training (demonstration downsampling):

- *Action Downsampling (Action-D)*: uniformly downsample the predicted action chunk A_t with a fixed factor n . Instead of performing actions a_1, a_2 , etc., here the robot performs actions a_1, a_{1+n} , etc. This

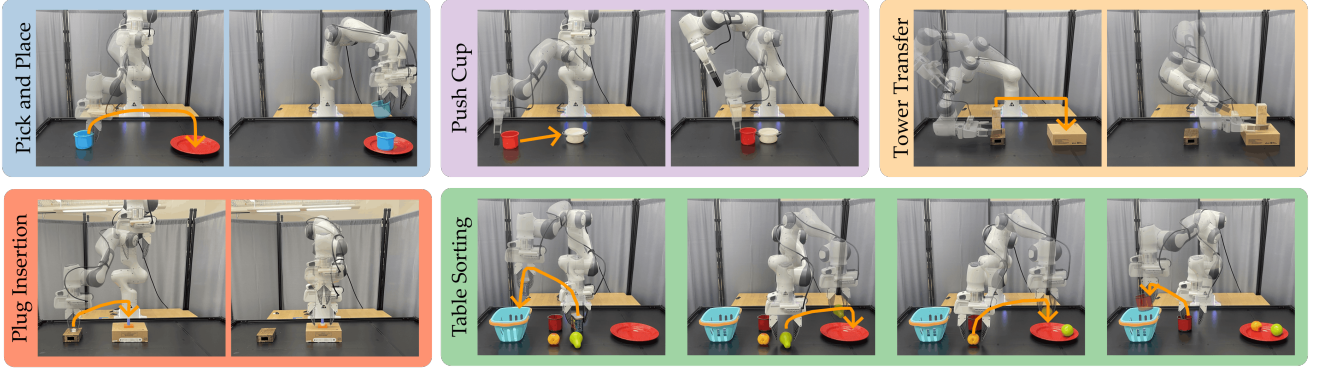


Fig. 2: Real-world manipulation tasks evaluated in our experiments. The manipulated objects are randomly initialized in each task, and the target object location is fixed. (Top) We include short-horizon tasks that require object transport: *Pick and Place* and *Tower Transfer*, and non-prehensile manipulation in *Push Cup*. (Bottom) *Plug Insertion* is a challenging task requiring precise control and force-based interaction, and *Table Sorting* is a long-horizon task where the robot sorts fruits on a plate and containers in a basket.

shortens the task into fewer steps and thereby accelerates the robot’s execution.

- *Demonstration Downsampling (Demo-D)*: uniformly downsample each demonstration by factor n . Instead of training on trajectory $\xi = \{(o_1, x_1), (o_2, x_2), \dots\}$, here the robot trains with $\xi = \{(o_1, x_{1+n}), (o_{1+n}, x_{1+2n}), \dots\}$. Downsampling makes the demonstration shorter, resulting in a faster learned policy.

Evaluation Setup. We perform all experiments on a Franka Emika robot arm on a tabletop setup as shown in Figure 2. We collect all demonstrations using a GELLO [30] to teleoperate the arm at 20 Hz. For each timestep, we record the robot’s joint position, including the gripper state x , the commanded joint and gripper state a_t , and RGB images y from three RealSense D435 cameras. Two of the cameras are fixed in the environment, and one is directly mounted on the robot’s end-effector. Each paradigm is tested on the following manipulation tasks:

- *Pick and Place*: The robot grasps a blue cup located on the table and places it on top of a red plate.
- *Push Cup*: The robot pushes a red cup to the nearest side of a white bowl while maintaining contact only with the outside of the cup.

4.1 Acceleration During Execution

Given a trained policy on the original dataset, the first strategy we consider is speeding up its execution. We take a policy π and modify its action outputs so that the

robot completes the task in less time. The way we modify these actions depends on the policy structure. Referring back to Equation (1), our diffusion policy outputs a sequence A of actions, where each action is a commanded joint position and gripper state. We can accelerate the robot’s motion by *skipping* actions within the sequence, i.e., downsampling A by a fixed rate n . After downsampling, the new sequence A' has a reduced horizon; thus, the execution time of a single prediction decreases linearly as n increases. Since the prediction horizon has fixed length H , the downsampling rate n is constrained to at most executing one action ($n \leq H - 1$). Note that omitting parts of the predicted actions makes the scheduled trajectory lose fine-grained details, and the low-level control has to apply larger control signals to track the desired states in A' , due to the increased distance in consecutive actions. Considering that completing a task may not require the robot to visit all states that the human demonstrated, this uniform acceleration could potentially speed up task execution.

Figure 3 compares the performance of *Normal Speed* against test-time downsampling (*Action-D*) at rates $n = 2$ and $n = 4$. As expected, the total time decreases as n increases. However, the performance of the learned policy degrades, particularly in tasks where the robot needs consistent, fine-grained motions. We observed that uniformly downsampling predicted action chunks often prevented the robot from executing actions critical for task completion. For example, when moving to push the cup, the robot needed to align its gripper behind the cup before initiating the push motion; if the accelerated actions A' skip these steps, then the robot failed the task. A more fundamental issue, however, was the mismatch between the robot’s speed

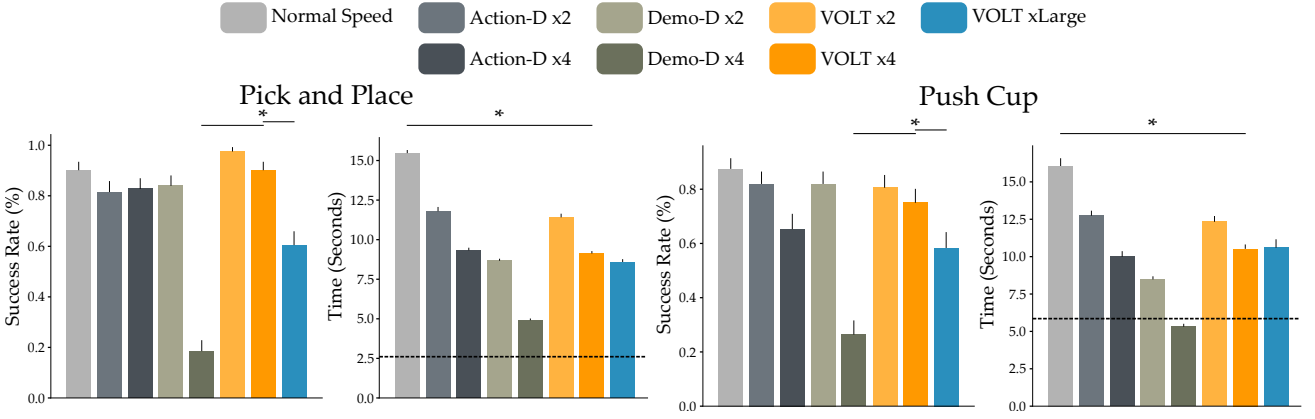


Fig. 3: Experimental results for Section 4. Results are averaged across 3 training runs. *Normal Speed* is trained with the default dataset. In *Action-D* action sequences are downsampled at run time to accelerate task execution. By contrast, in *Demo-D* actions are downsampled at training time to emulate a human with faster teaching motions. *VOLT* is similar to *Demo-D*, but it uses a VLM to determine which segments of the trajectory should or should not be downsampled. The downsampling rate is captured by n ; i.e., in $\times 2$ the robot skips every other action. We find that speeding up the policy during execution (*Action-D*) degrades performance as n increases, since the policy misses critical actions (e.g., aligning the grippers). Similarly, speeding up the entire policy during training (*Demo-D*) lowers the success rate. *VOLT* is able to maintain its success by only accelerating the parts of the task that do not require precise motion. However, the performance of *VOLT* still degrades if the human increases n to a level where the low-level controller is unable to track the commanded actions (i.e., there are still practical limitations). The dotted lines in Time denote the lower bound for *VOLT* if the robot could instantaneously complete the accelerated segments. An * denotes statistical significance, and error bars indicate standard error.

during training and testing. The observation history $O_t = \{o_{t-M}, \dots, o_t\}$ used to train π was obtained at the human’s teaching speed. When we accelerate during execution, the robot collects observations O with larger motions between images, pushing the robot’s policy farther out-of-distribution as n increases. This mismatch between behaviors the policy has been trained upon and the behaviors expected at test time limits acceleration during execution and degrades task performance.

4.2 Accelerating During Training

With these issues in mind, we next explore methods that downsample the human’s offline dataset. The core idea here is that — by reducing the number of datapoints — we effectively make the human’s demonstration faster. Policies trained on the accelerated dataset expect rapid movement during execution, and thus do not have the out-of-distribution challenge faced in Section 4.1. In practice, we perform downsampling by dividing a single trajectory into n non-overlapping versions. The first version is $\xi^1 = \{(o_1, a_1), (o_{1+n}, a_{1+n}), \dots\}$, the second version is $\xi^2 = \{(o_2, a_2), (o_{2+n}, a_{2+n}), \dots\}$, and so on. Similar to be-

fore, we train new policies with $n = 2$ and $n = 4$, and test their performance in Figure 3.

We find that *Demo-D* has the potential to significantly reduce execution time, and performs the task faster than equivalent versions of *Action-D*. But this speed comes at a cost: increasing to $n = 4$ sharply reduces the success rate, significantly under-performing the alternatives. Observing the robot’s motion, we find that *Demo-D* fails because the robot is trying to complete segments of the task *too rapidly*. For example, in the pick and place task the robot often misses the cup, and in the pushing task the robot often knocks the cup instead of sliding it across the table. The policy tries to address these mistakes — but by the time the robot moves to fix errors, the end-effector is already far from the cup and out-of-distribution. Accelerating during training has potential, but naively downsampling the entire motion results in critical failures.

4.3 VOLT: Vision and Language Trajectory Segmentation

Now that we have a method for accelerating the robot’s policy, we will determine *when* to speed up the robot’s motion. Our intuition is that the videos included within

the demonstration provide the context necessary for segmenting the trajectory. For instance, through these videos we can separate between a robot moving close to a cup unintentionally, and a robot moving towards a cup to grasp it. Based on this insight, we propose **VOLT: Vision and Language Trajectory Segmentation** to separate each demonstration into trajectory segments of types (1) *maintain-speed*: subtasks requiring precision or dexterity, and (2) *speed-up*: simpler motions that a robot can exploit to complete a task in less time than the human demonstrator.

The VLM’s input (V, p) consists of an entire video demonstration $V = \{y_0, y_1, \dots, y_T\}$ along with a text prompt p describing the trajectory segmentation objective with general task-invariant guidelines. We provide a few in-context examples of diverse tasks (excluding the target task) to enhance reasoning and encode optimal strategies. The model outputs an ordered set of segments $\{(\tau_0, l_0), \dots, (\tau_k, l_k)\}$, where τ denotes the start and end index for the segment, and l is the label.

We specifically leverage Qwen3-VL-32B-Instruct-FP8 [2] to autonomously generate the labels for the entire dataset. Details of the VLM implementation, including prompts and sampling hyperparameter are detailed in Appendix A.2. We evaluate the computational cost for segmenting for all the tasks shown in Figure 2 using a single H200 GPU. We observe that VOLT generates the labels for all 250 demonstrations in our dataset in ~ 32 minutes with batch inference, where the longest demonstrations average 1000 frames. To further assess VOLT’s applicability to large-scale datasets and longer horizon tasks, we label the 10 longest demonstrations from DROID [17], where each selected demonstration contains over 3500 frames: in this case, VOLT requires 12.5 minutes to segment all demos. We note that for demonstrations containing 3000 frames is best to either downsample the video or separate it into chunks for parallel segmentation over shorter context windows.

During training, the robot now has access to a dataset $\mathcal{D}$ with samples $\{(x, y, l)\}$ and user’s chosen downsampling factor n . The robot then trains its policy on the resampled dataset. We apply downsampling only for observations o_t when label $l_t = \textit{speed-up}$. To directly compare against the previous approaches, we use downsampling factors $n = 2$ and $n = 4$. Our results compare VOLT to the alternatives in Figure 3. For the pick and place task we find that models trained with VOLT maintain or achieve a higher success rate than *Normal Speed*, while for the cup pushing task there is a slight decrease in performance, usually due to errors in gripper placement behind the cup. As expected, only speeding up in the segments identified by VOLT yield policies that are slower than naively downsampling the

entire trajectory. But this approach ensures the robot completes the task faster than the *Normal Speed* policy without sacrificing its success rate.

4.4 What Limits VOLT’s Acceleration?

Our previous experiments show that we can leverage VOLT to accurately estimate when to accelerate. But what limits the downsampling factor n ? Here we discuss the practical factors that prevent designers from achieving even faster motions.

Policy Inference Delay. The robot’s high-level policy π converts its observations into actions. To prevent unintended pauses during evaluation, these actions must be readily available to the robot. However, when using diffusion policies the inference process — i.e., reasoning over images and generating the action sequence — is computationally expensive and typically exceeds the high-level control frequency. We mitigated this challenge by performing policy inference and action execution asynchronously. But there still remains a fundamental restriction on the control frequency f as a function of the inference delay $t_{\text{inference}} < \frac{H}{f}$. Although increasing horizon H allows us to also increase f , executing longer action horizons prevents the robot from reacting to changes in the environment.

Open-Loop vs. Closed-Loop. When the robot’s pacing matches the human demonstrator, it can consistently update its actions in response to errors, and make corrections to improve its performance. But if the robot maintains the same control frequency — and accelerates its motions — then the robot may have large errors before it queries its policy for a correction. Taken to an extreme, if we increase n such that each segment is just the start and goal waypoints, then the robot loses its ability to reason over any changes in the environment. Designers must tune n to trade-off between faster, open-loop execution and slower, closed-loop corrections.

Experiment Results. To test these concepts we increased n until the low-level controller was unable to track the reference positions in *A*. The results are summarized in Figure 3 as *VOLT xLarge*. Compared to the VOLT alternatives, *VOLT xLarge* has significantly lower success rates in both tasks (decreasing performance for *Pick and Place* by 29.7% and *Push Cup* by 29.2%). Interestingly, the completion time with *VOLT xLarge* is not significantly lower: this is because the robot’s policy makes several mistakes due to tracking errors, and takes added time to redo or correct its motions. Example failures include missing the cup when trying to grasp, knocking the cup over by mistake, or

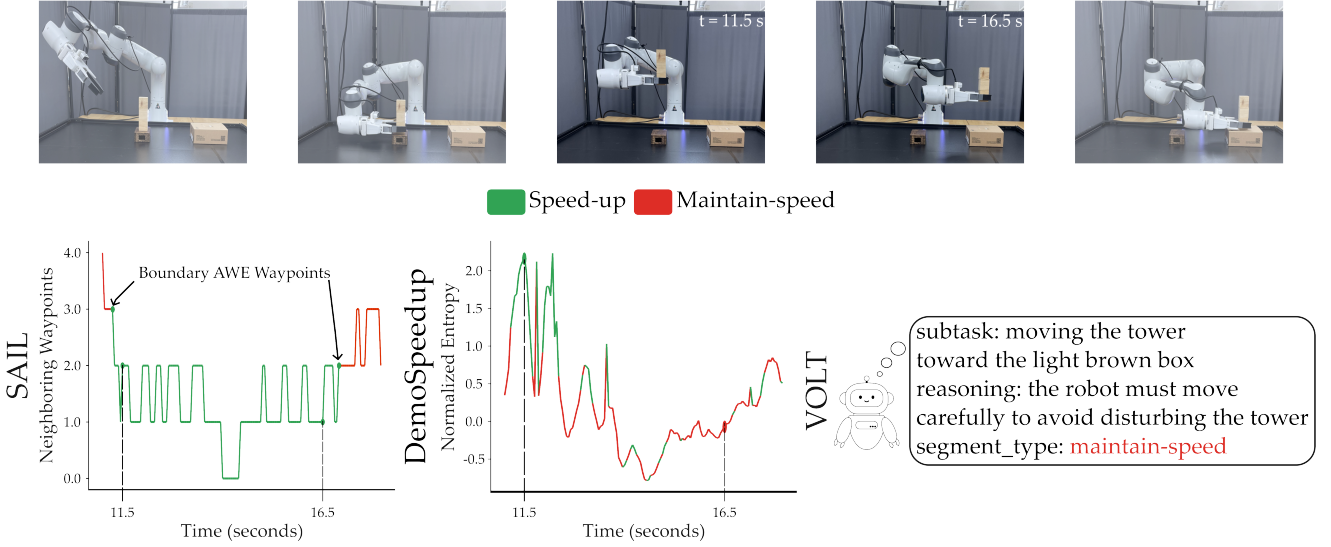


Fig. 4: Example segments for transporting the block tower in the Tower Transfer task. Ideally, the robot should maintain speed during the segment from 11.5s – 16.5s, since speeding up causes the blocks to fall over. *SAIL* labels the entire sequence as *speed-up* because a straight-line motion has low complexity. *DemoSpeedup* oscillates between modes while focusing on the entropy in the human’s demonstrations. Both of these approaches cause the tower to fall apart during transfer. *VOLT* correctly recognizes that speeding up could lead to failure, and accordingly labels the segment as *maintain-speed*.

pushing the cup at the wrong angle. Overall, we conclude that the designer cannot simply increase n until they reach their robot’s physical or control limits. The time delay associated with the policy imposes constraints that stack with any delays in the sensors (e.g., cameras are often limited to 60 frames per second).

5 How Does VOLT Compare to Alternatives?

In this section we compare *VOLT* against imitation learning approaches designed to accelerate policy execution. We focus on comparing both the performance and speedup magnitude over a standard policy trained on the human’s demonstrations. We utilize the same tabletop setup as in Section 4 and evaluate additional short-horizon and long-horizon tasks (see Figure 2).

Baselines. We compare *VOLT* to the two most similar baselines: *DemoSpeedup* and *SAIL*. Both of these approaches have the same general structure as *VOLT*, in that they use a high-level controller π to select the desired positions and a low-level controller to reach those positions as quickly and accurately as possible. The differences are explained below:

1. *DemoSpeedup*: Learns a proxy policy on the original dataset to estimate the conditional action entropy of each demonstration [14]. The authors employ hierarchical density-based spatial clustering (HDB-

SCAN) [21] to identify low-entropy clusters representing regions where the robot requires precise motion, while high-entropy outliers represent motions that do not require precision.

2. *SAIL*: Extracts waypoints from each demonstration using AWE [25] and estimate motion complexity by performing DBSCAN [12] clustering on the selected waypoints. Similar to *DemoSpeedup*, *SAIL* identifies clustered regions as precise motion and outliers as the non-precision regions. In the original implementation these labels are predicted at runtime to modulate control frequency. For a direct comparison to our approach, we use the generated labels to downsample the dataset before training.

Tasks. In this section, we evaluate all approaches on five manipulation tasks. The expected progression for each task is shown in Figure 2. Both *Pick and Place* and *Push Cup*, were introduced in Section 4. Below, we describe the goal for the new manipulation tasks:

- *Tower Transfer*: The robot transfers a tower of freely stacked wooden blocks to a new platform.
- *Plug Insertion*: The robot picks a plug and inserts it into a socket that is fixed in place. We test on assembly ID #00186 from the *AutoMate* assembly benchmark [27].
- *Table Sorting*: A long-horizon task with fruits and empty containers are scattered on a table. The

robot’s goal is to sort the fruits on a plate and the containers in a basket.

Training. We collect demonstrations using the robot arm setup from Section 4. For each short-horizon task we collect ~ 50 demonstrations, and 100 demonstrations for the long-horizon *Table Sorting* task. Across demonstrations, we randomize the initial position of the manipulated objects within a pre-defined region, and keep the target object positions fixed. The same training data is provided to *VOLT* and the baselines. We then use each separate algorithm to determine which segments of these demonstrations to downsample. Once the segments are assigned by *VOLT*, *DemoSpeedup*, and *SAIL*, we then train separate models with identical network architectures and hyperparameters for every method to assess how the label affect policy speedup and performance. For *Pick and Place*, *Push Cup*, and *Tower Transfer*, we apply a downsampling rate of $\times 2$ and $\times 4$ for *maintain-speed* and *speed-up* segments, respectively. While for *Plug Insertion* and *Table Sorting*, we apply $\times 1$ and $\times 3$ downsampling rates..

5.1 How Accurately Does VOLT Segment Demonstrations?

The key difference between *VOLT* and the baselines is how they segment the trajectory. When comparing methods, we focused on how accurately each approach decomposes the human’s demonstrations into labeled subtasks: $l = \textit{speed-up}$ for simple segments and $l = \textit{maintain-speed}$ for precise motions. In Figure 4 we show an example for *Tower Transfer*. Intuitively, the robot should move quickly to pick up the blocks, and then maintain the demonstrated speed while transferring the tower. Given the exact same demonstration, *SAIL* incorrectly speeds up the entire motion of transferring the tower, while *DemoSpeedup* alternates between accelerating and maintaining speed; only *VOLT* correctly recognizes that the transfer motion should not be accelerated. Note that this labeling is not possible with alternatives [18], since they rely on distance features — and here the relevant feature is how the block tower may collapse.

In general, while we observe that *SAIL* correctly labels gripper actuation as *maintain-speed* from the heuristics of AWE, this method more aggressively categorizes regions as *speed-up* compared to the alternative methods. On the other hand, *DemoSpeedup* is overly conservative and labels most parts of the task as *maintain-speed*; this most likely occurs because a fixed goal position for the manipulated object produces a lower entropy value around these repetitive movements.

While *DemoSpeedup* is more conservative, the entropy estimation often leads to unexpected switching between *maintain-speed* and *speed-up* for precise segments.

5.2 Does VOLT Successfully Accelerate Policy Execution?

Our experimental results are summarized in Figure 5. A one-way ANOVA showed that policies trained with *VOLT* complete tasks significantly faster than *Normal Speed* across all tasks ($p < 0.001$ for all). *VOLT* achieved up to $\times 2.57$ speedup in the *Pick and Place* task, and an average $\sim \times 2.13$ over *Normal Speed* without negatively impacting performance. In particular, *VOLT* significantly outperforms training with the original demonstration in *Table Sorting* ($p < 0.05$), where *VOLT* exhibits a more consistent strategy of handling one object at a time, while *Normal Speed* often switches which object to approach while moving towards them. We also observe a performance gain on *Tower Transfer* and *Plug Insertion* tasks. Consistent with prior work, we attribute this increase in success rate to the reduced decision-making horizon of downsampled demonstrations [25], which reduces compounding errors.

As compared to the baselines *VOLT* consistently obtains the second largest speedup. *SAIL* is the fastest — because it assigns the most *speed-up* segments — but its overly aggressive choices cause the robot to make mistakes that detrimentally affect performance in *Pick and Place* and *Push Cup*. By contrast, *DemoSpeedup* most frequently labels segments as *maintain-speed*, resulting in a slower policy with similar success rates. Overall, our results suggest that *VOLT* is capable of correctly estimating when to accelerate.

5.3 Ablation Study

The key components in our method to enable VLMs to segment demonstrations are task-invariant guidelines, a few in-context examples, and the elicited reasoning. We ablated them on the *Plug Insertion* task.

- *No Examples*: We test our method without in-context examples to test the VLM’s ability to identify segments without including visual guidance.
- *No Examples + Simple Prompt*: We additionally remove the task-invariant textual guidance and generated reasoning to evaluate the VLM’s baseline performance.
- *Smaller VLM*: We generate labels with our proposed configuration and only change the VLM to Qwen3-VL-8B-Instruct-FP8 to test how VLM size affects performance.

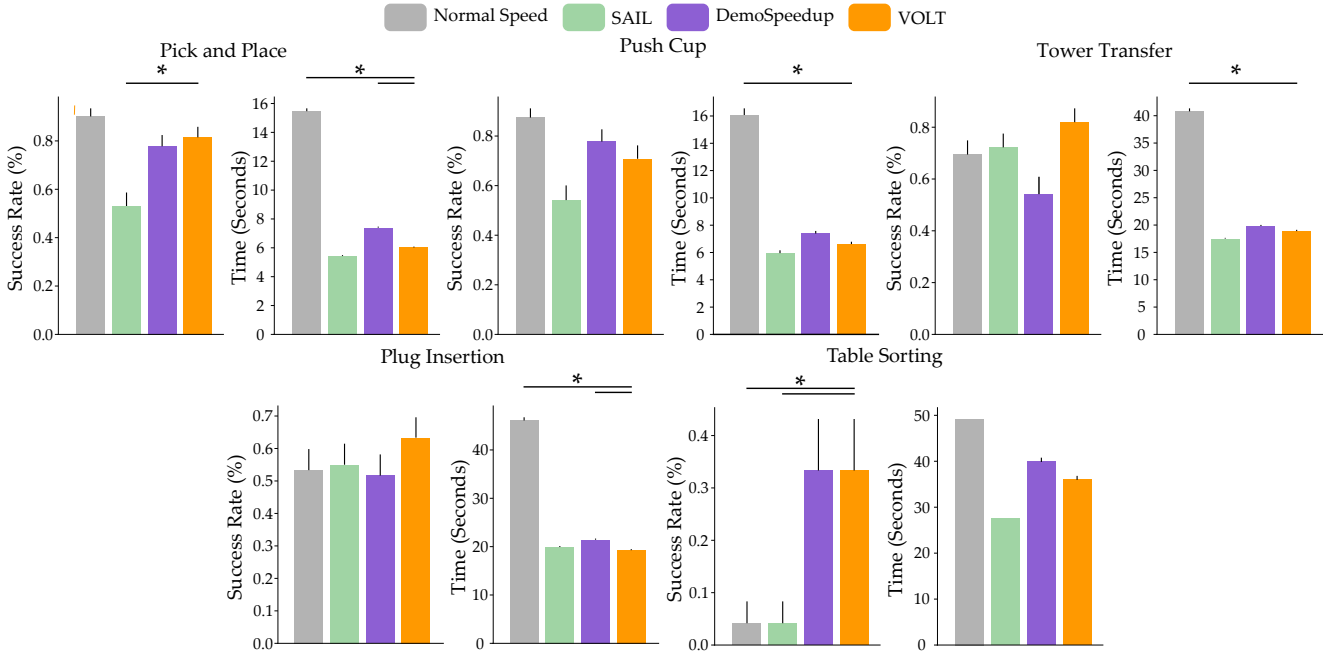


Fig. 5: Experimental results for Section 5. We compare VOLT against state-of-the-art baselines across five manipulation tasks: *Pick and Place*, *Push Cup*, *Tower Transfer*, *Plug Insertion*, and *Table Sorting* (see Figure 2). All methods use the same network architecture and downsampling rate. We report results for a single model in *Table Sorting* and average results over 3 training runs for the short-horizon tasks. In our results *SAIL* overestimates *speed-up* segments (leading to faster but less successful policies), while *DemoSpeedup* often misses *speed-up* segments (leading to slower but consistent policies). We find that *VOLT* balances between these extremes: significantly reducing task completion time, and achieving the second largest speedup across all tasks while maintaining similar performance to *Normal Speed*. An * denotes statistical significance, and error bars indicate standard error.

Table 1: Ablation study results. *Normal Speed* policies achieve 53.33% success rate and take 46.1 seconds to complete the task.

Method	Success Rate (%)	Time (s)
No Examples	48.33	19.27
No Examples + Simple Prompt	31.67	18.93
Smaller VLM	46.67	17.88
VOLT (Ours)	63.33	19.27

Results. We find that for *No Examples*, removing the in-context examples results in two main differences in the estimation of segment boundaries for some demonstrations compared to our full configuration. Without examples, the VLM becomes more conservative for *maintain-speed* segments; specifically, in the generated labels, the model classifies the alignment of the gripper and the target object to start sooner than with *VOLT*. Additionally, there are cases where the model will assign more *speed-up* segments mainly by deciding to speed up as soon as the gripper closes or the robot places the socket. As shown in Table 1 this differences result in policies that are significantly faster than

Normal Speed with a slight negative impact to the success rate. In the case of only providing the model with the goal of the segmentation task *No Examples + Simple Prompt*, without the use of any visual or textual guidance, we observe that, similar to the *No Examples* variant, the model does recognize the correct order of events that occur in the video demonstration, and it is capable of then accordingly. However, removing all sources of guidance leads to the VLM poorly estimating the end of each segment, and typically estimating the end of each subtask earlier than its actual timing, which causes the resulting policy to speed up wrongfully during grasping. Consequently, this variant negatively impacts task performance, causing a significant drop in success rate compared to *Normal Speed* policies.

The segmentation generated by the smaller VLM variant results in labels that allocate more frames to *speed-up* segments and reduce the boundaries for *maintain-speed* segments. Notably, this model failed to recognize the correct frame where the robot completes an interaction with an object, for example, during the grasping of the socket, or when the robot finishes insert-

ing the socket into the plug. As compared to the larger capacity model, the smaller VLM has a more aggressive segmentation style, which results in faster policies at the cost of task success rate. Overall, we find that each design choice significantly contributes to balancing the speed-performance trade-off for efficient policies.

6 Practical Considerations

We summarize the key design choices identified in this work to effectively implement faster-than-demonstration policies in practice.

- **Visual cues are necessary.** While utilizing the robot’s information to segment demonstrations can be effective, throughout our experiments in Section 5, we demonstrate that video reasoning is crucial to identify when to *maintain-speed* for simple but precise motions (e.g., transferring the tower).
- **Eliminate sensor-inference delays.** Effective faster-than-demonstration policies require the robot to always have actions available to execute, but synchronous inference introduces unintended pauses. As described in Section 4, we instantiate diffusion policies to predict action chunks and perform asynchronous inference to eliminate these delays. In addition, we apply EAG to ensure consistency between asynchronously predicted actions. We recommend that practitioners utilize asynchronous inference when the effective policy frequency is slower than the intended high-level frequency, along with action consistency methods [1, 7].
- **Reduce the receding control horizon.** As the robot operates at greater speeds, it will become less reactive. Our results from *VOLT xLarge* in Section 4.4 suggest that instead of executing a long sequence of open-loop actions, it is better to reduce the policy action horizon and query π more frequently to correct potential errors. We can then pass the output actions A to the low-level controller as soon as they are updated.
- **Optimizing the low-level controller.** In our hierarchical paradigm, the policy sets goal positions at a slower frequency, and the low-level controller tracks them via velocity or torque commands. Tuning low-level gains for fast, accurate tracking is essential to maximize acceleration to prevent the policy from reaching out-of-distribution states.
- **Smooth transition between fast and precise segments.** Abrupt changes in acceleration cause

motion jittering, which prevents the robot from reliably executing tasks near the boundaries of segments. In our experiments, we apply action interpolation near these boundaries. In practice, we recommend downsampling rates starting with 2x-4x, which have been tested to perform consistently well.

7 Conclusion

In this paper, we presented VOLT, a data augmentation framework to speed up task execution without degrading task performance. Our approach recognizes which segments of a task a robot can complete faster than demonstrated, and segments where the robot should operate similarly to the demonstrator. Specifically, we identify these segments based on the context of the task and the images of the environment by leveraging the video understanding capabilities of VLMs. We then trained accelerated robot policies by downsampling the identified regions at different rates. Our experiments suggest that this approach enables robots to execute task more efficiently by increasing execution speed, while maintaining similar performance to models trained on the original dataset.

Limitations and Future Work. Our results suggest that VOLT is a step towards learning policies that balance between task performance and execution speed. However, our current implementation requires downsampling rates to be manually defined and applied uniformly. Future work should automate this process, possibly guided by the VLM’s own uncertainty, employing trajectory compression algorithms, or a hybrid approach combining VOLT with similar methods that have access to the low-level robot information.

8 Declarations

Funding. This work was supported in part by Mitsubishi Electric Research Laboratories and the work of authors R. Ramirez Sanchez, D. Evans, and D. Losey was also funded in part by the National Science Foundation (NSF) under Grant No. #2129201.

Conflict of Interest. The authors declare that they have no conflicts of interest.

Author Contribution. R.R.S.: Conceptualization, Investigation, Methodology, Data Collection, Experimentation, Writing - original draft. D.E.: Data Collection, Experimentation. D.L.: Conceptualization, Supervision, Funding Acquisition, Writing - review and editing. S.J.: Conceptualization, Supervision, Methodology, Funding Acquisition, Writing - review and editing.

References

1. Arachchige, N.R., Chen, Z., Jung, W., Shin, W.C., et al.: SAIL: Faster-than-demonstration execution of imitation learning policies. In: Conference on Robot Learning (2025)
2. Bai, S., Cai, Y., Chen, R., Chen, K., et al.: Qwen3-VL technical report. arXiv preprint arXiv:2511.21631 (2025)
3. Belkhale, S., Cui, Y., Sadigh, D.: Data quality in imitation learning. In: Advances in Neural Information Processing Systems (2023)
4. Belkhale, S., Cui, Y., Sadigh, D.: HYDRA: Hybrid robot actions for imitation learning. In: Conference on Robot Learning (2023)
5. Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, et al.: GR00T N1: An open foundation model for generalist humanoid robots. arXiv preprint arXiv:2503.14734 (2025)
6. Black, K., Brown, N., Driess, D., Esmail, A., et al.: π_0 : A vision-language-action flow model for general robot control. Robotics: Science and Systems (2025)
7. Black, K., Ren, A.Z., Equi, M., Levine, S.: Training-time action conditioning for efficient real-time chunking. arXiv preprint arXiv:2512.05964 (2025)
8. Chen, A.S., Lessing, A.M., Liu, Y., Finn, C.: Curating demonstrations using online experience. Robotics: Science and Systems (2025)
9. Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., Song, S.: Diffusion policy: Visuomotor policy learning via action diffusion. The International Journal of Robotics Research (2025)
10. Clark, J., Mirchandani, S., Sadigh, D., Belkhale, S.: Action-free reasoning for policy generalization. In: ICRA Workshop on Foundation Models and Neuro-Symbolic AI for Robotics (2025)
11. Dai, Y., Ramirez Sanchez, R., Jeronimus, R., Sagheb, S., Nunez, C.M., Nemlekar, H., Losey, D.P.: CIVIL: Causal and intuitive visual imitation learning. arXiv preprint arXiv:2504.17959 (2025)
12. Ester, M., Kriegel, H.P., Sander, J., Xu, X., et al.: A density-based algorithm for discovering clusters in large spatial databases with noise. In: Knowledge Discovery and Data Mining (1996)
13. Gao, T., Nasiriany, S., Liu, H., Yang, Q., Zhu, Y.: PRIME: Scaffolding manipulation tasks with behavior primitives for data-efficient imitation learning. IEEE Robotics and Automation Letters (2024)
14. Guo, L., Xue, Z., Xu, Z., Xu, H.: DemoSpeedup: Accelerating visuomotor policies via entropy-guided demonstration acceleration. In: Conference on Robot Learning (2025)
15. Jones, J., Mees, O., Sferrazza, C., Stachowicz, K., Abbeel, P., Levine, S.: Beyond Sight: Finetuning generalist robot policies with heterogeneous sensors via language grounding. In: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA). Atlanta, USA (2025)
16. Kawaharazuka, K., Oh, J., Yamada, J., Posner, I., Zhu, Y.: Vision-Language-Action Models for Robotics: A review towards real-world applications. IEEE Access (2025)
17. Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., et al.: DROID: A large-scale in-the-wild robot manipulation dataset. In: RSS 2024 Workshop: Data Generation for Robotics (2024)
18. Kim, B., Pahk, J., Lee, C., Kim, J., Lee, J., et al.: ES-PADA: Execution speedup via semantics aware demonstration data downsampling for imitation learning. arXiv preprint arXiv:2512.07371 (2025)
19. Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., et al.: OpenVLA: An open-source vision-language-action model. In: Conference on Robot Learning (2025)
20. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, et al.: Efficient memory management for large language model serving with pagedattention. In: Proceedings of the 29th symposium on operating systems principles (2023)
21. McInnes, L., Healy, J., Astels, S., et al.: hdbscan: Hierarchical density based clustering. Journal of Open Source Software (2017)
22. Nam, T., Hwang, S.J.: SpeedAug: Policy acceleration via tempo-enriched policy and RL fine-tuning. arXiv preprint arXiv:2512.00062 (2025)
23. Parekh, S., Nemlekar, H., Losey, D.P.: Towards balanced behavior cloning from imbalanced datasets. Autonomous Robots (2026)
24. Ramirez Sanchez, R., Nemlekar, H., Sagheb, S., Nunez, C.M., Losey, D.P.: RECON: Reducing causal confusion with human-placed markers. In: IEEE/RISJ International Conference on Intelligent Robots and Systems (2025)
25. Shi, L.X., Sharma, A., Zhao, T.Z., Finn, C.: Waypoint-based imitation learning for robotic manipulation. In: Conference on Robot Learning (2023)
26. Song, J., Meng, C., Ermon, S.: Denoising diffusion implicit models. In: International Conference on Learning Representations (2021)
27. Tang, B., Akinola, I., Xu, J., Wen, B., et al.: AutoMate: Specialist and generalist assembly policies over diverse geometries. Robotics: Science and Systems (2024)
28. Tsuji, T., Kato, Y., Solak, G., Zhang, H., Petrič, T., Nori, F., Ajoudani, A.: A survey on imitation learning for contact-rich tasks in robotics. The International Journal of Robotics Research (2025)
29. Wang, C., Fan, L., Sun, J., Zhang, R., Fei-Fei, L., Xu, D., Zhu, Y., Anandkumar, A.: MimicPlay: Long-horizon imitation learning by watching human play. In: Conference on Robot Learning (2023)
30. Wu, P., Shentu, Y., Yi, Z., Lin, X., Abbeel, P.: GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (2024)
31. Xu, H., Zhan, X., Yin, H., Qin, H.: Discriminator-weighted offline imitation learning from suboptimal demonstrations. In: International Conference on Machine Learning (2022)
32. Yuan, D.D., Zhao, T.Z., Burns, K., Finn, C.: Speedtuning: Speeding up policy execution with lightweight reinforcement learning. In: IEEE International Conference on Robotics and Automation (2025)
33. Zare, M., Kebria, P.M., Khosravi, A., Nahavandi, S.: A survey of imitation learning: Algorithms, recent developments, and challenges. IEEE Transactions on Cybernetics (2024)
34. Zhao, Q., Lu, Y., Kim, M.J., Fu, Z., Zhang, Z., Wu, et al.: CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In: Proceedings of the Computer Vision and Pattern Recognition Conference (2025)
35. Zhao, T.Z., Kumar, V., Levine, S., Finn, C.: Learning fine-grained bimanual manipulation with low-cost hardware. Robotics: Science and Systems (2023)
36. Zhu, Y., Jiang, Z., Stone, P., Zhu, Y.: Learning generalizable manipulation policies with object-centric 3d representations. In: Conference on Robot Learning. Proceedings of Machine Learning Research (2023)

A Appendix

A.1 Policy Training Hyperparameters

In Table 2 we list the hyperparameter configuration for the diffusion policy implementation shared across our method and baselines.

Table 2: Hyperparameter for diffusion transformer instantiation and training.

Hyperparameter	Value
Policy Architecture	
Observation Horizon	2
Prediction Horizon	32
Action Horizon	16
Attention Layers	8
Attention Heads	4
Embedding Dimension	256
Embedding Dropout	0.0
Attention Dropout	0.3
Future Action Conditioning	
Future Action Horizon	4
Conditioning Probability	0.3
Noise Scheduler	
Algorithm	DDIM
Beta Schedule	squaredcos_cap_v2
Training Iterations	100
Inference Iterations	10
Exponential Moving Average	
Inverse Gamma	1.0
Power	0.75
Training Configuration	
Image Size	[224, 224]
Batch Size	64
Epoch	1000
Learning Rate	$1e - 4$
Optimizer	AdamW
Weight Decay	$1e - 6$
LR Scheduler	Cosine linear warmup
Warmup steps	1000

A.2 VLM Implementation Details

We instantiate the VLM using vLLM [20] on a single H200. The model is loaded with a context window of 262k tokens and prefix caching for in-context examples and system prompt. In Figure 6 and Figure 7, we show the system and user prompt used in our experiments. In Table 3 we list the sampling hyperparameter values for reproducing our results, most parameters follow those suggested by Qwen’s developers, except for the temperature and presence penalty that were tuned for better performance on our task.

Table 3: VLM sampling hyperparameters for video segmentation.

Hyperparameter	Value
Random Seed	3047
Top P	0.8
Top K	20
Temperature	0.4
Repetition Penalty	1.0
Presence Penalty	0.5

System Prompt

You are a helpful assistant that generates labels for a given video of a robot performing a task. Your goal is to help the robot perform the task as fast as possible without causing it to fail by segmenting the video into different subtasks. You first need to analyze the video and then localize all the subtasks performed to complete the main task. For each subtask, you need to identify its start and end timestamps and classify the subtask into one of the valid segment types: speed-up, maintain-speed. Your generated response must help the robot increase its task efficiency while maintaining precise and critical movements.

Subtask Localization and Classification Guidelines:

- The robot should speed up whenever possible, this refers to cases that do not require delicate environment interactions.
- Maintain-speed only when it is necessary for the robot to work as demonstrated, mainly subtasks that require dexterity or precision.
- The robot is a physical system that cannot instantly change speed, you need to allocate enough frames on slower segments when transitioning between subtasks that do not share the same category to allow the robot to smoothly slow down and avoid jerky motions.
- Given direct grasping of an object, the robot has enough force to securely hold it regardless of speed.
- Fast speed will often lead to unexpected motion of objects not directly grasped by the gripper.
- Default the events where the robot is actively avoiding obstacles without an exaggerated motion, i.e., the robot barely clears the obstacle as precise segments.
- When the robot is done with the task and interacting with objects, the robot must quickly reach its final position.

Generation Rules:

- Provide a concise reasoning for which label is appropriate for each segment must be provided before making a decision.
- The list of segments must cover the entire video.
- Only leave one frame gaps between segments.
- All robot alignment subtasks must be categorized as precise.

The following is a non-exhaustive list of common subtasks:

- Moving towards objects
- Transporting objects
- Avoiding obstacles
- Grasping/Placing
- Insertion/Retrieval
- Pushing
- Pouring
- Wiping

This is not a complete list of possible subtasks. If any identified event is not included in this list, be sure to describe it accordingly.

The generated response must use the “mm:ss.ff” format for all timestamps. Final output format:

```
```json
```

```
[
 {
 "start_time": "mm:ss.ff",
 "end_time": "mm:ss.ff",
 "subtask": "concise subtask description",
 "reasoning": "reasoning of which segment_type should be used",
 "segment_type": "speed-up/maintain-speed"
 },
 ...
]
```

```
```
```

Fig. 6: System prompt for video segmentation

User Prompt

Given the current task description: <|video-description|>, help the robot complete the task faster than demonstrated in the video (00:00.00 - <|video-length|>)

Fig. 7: User prompt for video segmentation